

SKILL WEEK6

TASK-1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND_LENGTH 100

char history[MAX_HISTORY][MAX_COMMAND_LENGTH];
int count = 0;

/* Store a command */
void storeCommand(char command[])
{
    if (count < MAX_HISTORY)
    {
        strcpy(history[count], command);
        count++;
    }
    else
    {
        /* Remove the oldest command */
        for (int i = 0; i < MAX_HISTORY - 1; i++)
        {
            strcpy(history[i], history[i + 1]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }

    printf("Command stored successfully.\n");
}

/* Display command history */
void displayHistory()
{
    if (count == 0)
    {
        printf("History is empty.\n");
    }
}
```

```
        command[strcspn(command, "\n")] = '\0';

        if (strlen(command) == 0)
        {
            printf("Empty command cannot be stored.\n");
        }
        else
        {
            storeCommand(command);
        }
        break;

    case 2:
        displayHistory();
        break;

    case 3:
        retrieveCommand();
        break;

    case 4:
        validateConsistency();
        break;

    case 5:
        printf("Exiting program...\n");
        return 0;

    default:
        printf("Invalid choice.\n");
    }
}

return 0;
```

```
laasya@laasya: ~/skill week6 × + v
laasya@laasya:~$ cd week6
-bash: cd: week6: No such file or directory
laasya@laasya:~$ mkdir "skill week6"
laasya@laasya:~$ cd "skill week6"
laasya@laasya:~/skill week6$ nano task1.c
laasya@laasya:~/skill week6$ gcc task1.c -o task1
laasya@laasya:~/skill week6$ ./task1

===== COMMAND HISTORY BUFFER =====
1. Store Command
2. Display History
3. Retrieve Command
4. Validate Consistency
5. Exit
Enter your choice: 1
Enter command: ls
Command stored successfully.

===== COMMAND HISTORY BUFFER =====
1. Store Command
2. Display History
3. Retrieve Command
4. Validate Consistency
5. Exit
Enter your choice: 2

----- COMMAND HISTORY -----
1. ls

===== COMMAND HISTORY BUFFER =====
1. Store Command
2. Display History
3. Retrieve Command
4. Validate Consistency
5. Exit
Enter your choice: |
```

TASK2

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 5
#define MAX_ARGUMENTS 10
#define MAX_LENGTH 100

/* Structure to store one command */
typedef struct
{
    char command[MAX_LENGTH];
} Command;

/* Structure to store the complete pipeline */
typedef struct
{
    Command commands[MAX_COMMANDS];
    int count;
} Pipeline;

/* Add command to pipeline */
void addCommand(Pipeline *pipeline, char command[])
{
    if (pipeline->count < MAX_COMMANDS)
    {
        strcpy(pipeline->commands[pipeline->count].command, command);
        pipeline->count++;

        printf("Command added: %s\n", command);
    }
    else
    {
        printf("Pipeline capacity reached.\n");
    }
}

```

```
        printf("Empty command is not allowed.\n");
    }
    else
    {
        addCommand(&pipeline, command);
    }
    break;

case 2:
    displayPipeline(&pipeline);
    break;

case 3:
    validatePipeline(&pipeline);
    break;

case 4:
    executePipeline(&pipeline);
    break;

case 5:
    pipeline.count = 0;
    printf("Pipeline cleared.\n");
    break;

case 6:
    printf("Exiting program...\n");
    return 0;

default:
    printf("Invalid choice.\n");
```

```
laasya@laasya:~/skill week6$ nano task2.c
laasya@laasya:~/skill week6$ gcc task2.c -o task2
laasya@laasya:~/skill week6$ ./task2
```

```
===== PIPELINE MANAGEMENT =====
```

1. Add Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Clear Pipeline
6. Exit

Enter your choice: ls|sort

Invalid choice.

```
===== PIPELINE MANAGEMENT =====
```

1. Add Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Clear Pipeline
6. Exit

Enter your choice: Invalid choice.

```
===== PIPELINE MANAGEMENT =====
```

1. Add Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Clear Pipeline
6. Exit

Enter your choice: Invalid choice.

```
===== PIPELINE MANAGEMENT =====
```

```
===== PIPELINE MANAGEMENT =====
1. Add Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Clear Pipeline
6. Exit
Enter your choice: Invalid choice.

===== PIPELINE MANAGEMENT =====
1. Add Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Clear Pipeline
6. Exit
Enter your choice: Invalid choice.

===== PIPELINE MANAGEMENT =====
1. Add Command
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Clear Pipeline
6. Exit
Enter your choice: |
```

TASK-3 To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t pid1, pid2;

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    printf("Pipe created successfully.\n");

    /* Create first child process */
    pid1 = fork();

    if (pid1 == -1)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid1 == 0)
    {
        /* First child */

        printf("Child 1: Executing 'ls'\n");

        /* Close unused read end */
        close(pipefd[0]);
    }
}
```

```

        close(pipefd[1]);

        /* Redirect standard input from pipe */
        if (dup2(pipefd[0], STDIN_FILENO) == -1)
        {
            perror("dup2");
            exit(EXIT_FAILURE);
        }

        /* Close original read descriptor */
        close(pipefd[0]);

        /* Execute sort */
        execlp("sort", "sort", NULL);

        perror("execlp");
        exit(EXIT_FAILURE);
    }

    /* Parent process */

    /* Parent does not use the pipe */
    close(pipefd[0]);
    close(pipefd[1]);

    printf("Parent: Waiting for child processes...\n");

    /* Synchronize processes */
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Parent: Both child processes completed.\n");
    printf("Data flow test completed successfully.\n");

    return 0;
}

```

```

laasya@laasya:~/skill week6$ nano task3.c
laasya@laasya:~/skill week6$ gcc task3.c -o task3
laasya@laasya:~/skill week6$ ./task3
Pipe created successfully.
Child 1: Executing 'ls'
Parent: Waiting for child processes...
Child 2: Executing 'sort'
task1
task1.c
task2
task2.c
task3
task3.c
Parent: Both child processes completed.
Data flow test completed successfully.

```

TASK-4: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_PROCESSES 5

int main()
{
    int num_processes = 3;

    /*
     * Commands:
     * Process 1: ls
     * Process 2: sort
     * Process 3: wc -l
     */
    char *commands[MAX_PROCESSES][3] =
    {
        {"ls", NULL, NULL},
        {"sort", NULL, NULL},
        {"wc", "-l", NULL}
    };

    /*
     * For N processes, N-1 pipes are required.
     */
    int pipes[MAX_PROCESSES - 1][2];

    pid_t pids[MAX_PROCESSES];

    printf("==== MULTIPLE PIPE PROGRAM ====\n");
    printf("Number of processes: %d\n", num_processes);
    printf("Pipeline: ls | sort | wc -l\n\n");

    /*

```

```

        /*
        * Execute command
        */
        execvp(commands[i][0], commands[i]);

        perror("execvp");
        exit(EXIT_FAILURE);
    }
}

/*
* Parent closes all pipe descriptors
*/
for (int i = 0; i < num_processes - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

printf("\nParent: Waiting for all processes...\n");

/*
* Wait for all child processes
*/
for (int i = 0; i < num_processes; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("\nAll processes completed successfully.\n");
printf("All pipe descriptors cleaned up.\n");
printf("Long pipeline execution completed.\n");

return 0;
}

```

```

laasya@laasya:~/skill week6$ nano task4.c
laasya@laasya:~/skill week6$ gcc task4.c -o task4
laasya@laasya:~/skill week6$ ./task4
===== MULTIPLE PIPE PROGRAM =====
Number of processes: 3
Pipeline: ls | sort | wc -l

Pipe 1 created successfully.
Pipe 2 created successfully.

Parent: Waiting for all processes...
0

```